Case Studies

Data Engineer

Two example case studies about Azure data platforms and production data integrations.

HOW TO READ CASE STUDIES

These case studies go through how to approach common data platform challenges. The idea is to show how approach platform modernization and data integrations development.

CASE STUDIES

Modernizing a Legacy Data Platform into an Azure Lakehouse

Moving scattered source systems and old warehouse processes to an Azure lakehouse.

Production Data Integrations

Ingestion and transformation patterns for production: retries, schema changes, data quality and recovering from failures.

MODERNIZING A LEGACY DATA PLATFORM INTO AN AZURE LAKEHOUSE

Representative Scenario

Data is in several operational systems and in old warehouse. Its hard to modify pipelines and same logic is implemented in multiple places. Nobody sees data quality or why production jobs fail.

Approach

1. List important sources, data products, owners, access limits and freshness needs. Pick small and useful part to migrate first.
2. Use medallion style, Databricks and Azure Data Factory where they make sense.
3. Build ingestion, transformations, data models and automated quality checks.
4. Add access control, monitoring, CI/CD and infrastructure automation for releases.
5. Document who owns what and how to recover from failures.

Outcomes to Measure

- How much time it takes to add a new data source.
- Pipeline reliability, data freshness and how fast failures are noticed and fixed.
- Number of data quality issues.
- Running cost and performance.

Relevant Capabilities

Azure Databricks, Azure Data Factory, Apache Spark, Python, SQL, lakehouse architecture, data modeling, data quality, Terraform, CI/CD and production operations.

PRODUCTION DATA INTEGRATIONS

Representative Scenario

Data comes from APIs, databases and files. The ingestion jobs break when source is down, schema changes or run has to be repeated. It is hard to find out what went wrong when there are issues.

Approach

1. Before implementation, agree on the source contracts and owners how fresh the data needs to be.
2. Build ingestion with checkpoints and idempotent processing so retries do not create duplicates or skip data.
3. Handle schema changes and validate data so invalid or unexpected records are visible.
4. Add structured logs, metrics and alerts.
5. Automate tests and deployments.

Outcomes to Measure

- Successful runs, missing or duplicate records.
- Time to detect, find cause and recover when pipeline fails.
- Time to add a new source.
- Time to handle a schema change

Relevant Capabilities

Azure Data Factory, Azure Databricks, Python, SQL, Apache Spark, API and database integration, data quality, CI/CD, Terraform, monitoring and production operations.